EX-01

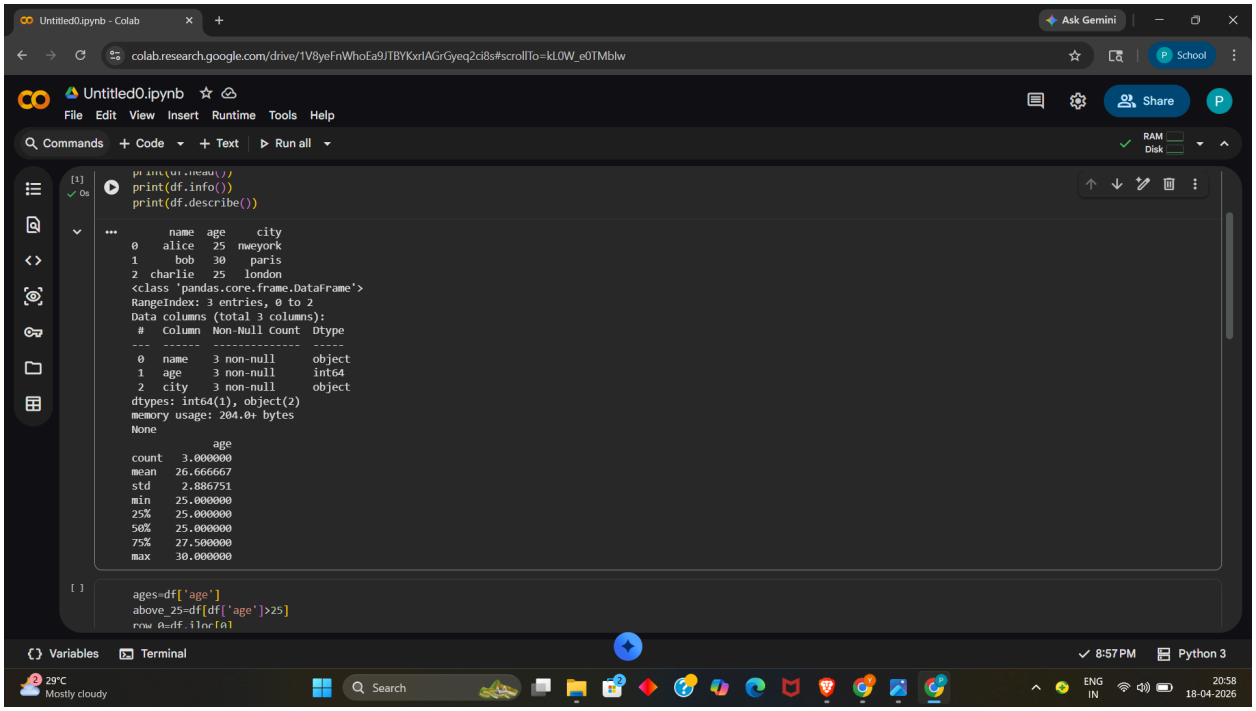
1.1numpy(library):-

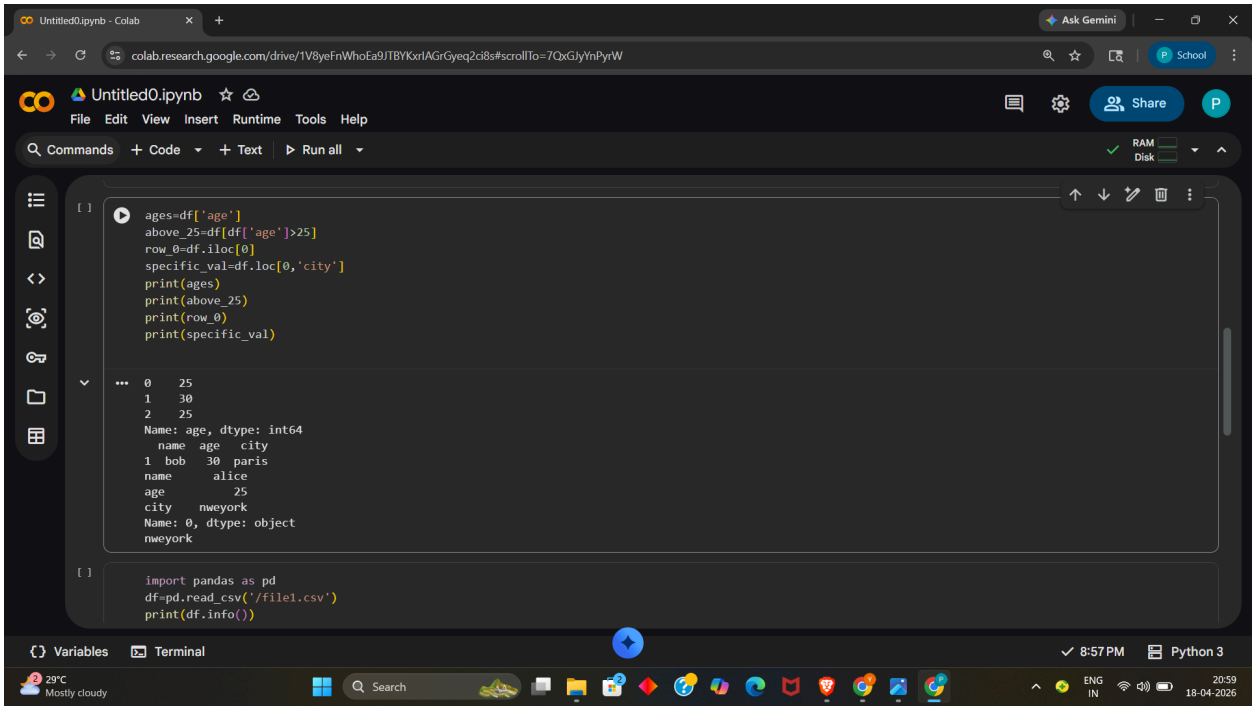
```
import numpy as np

# 1 Array Creation
a = np.array([1, 2, 3, 4, 5])
b = np.arange(1, 10, 2)
c = np.linspace(0, 1, 5)
d = np.zeros((2, 3))
e = np.ones((2, 2))
f = np.eye(3)
g = np.random.randint(1, 100, size=(3, 3))

print("Array a:", a)
print("Arange b:", b)
print("Linspace c:", c)
print("Zeros d:\n", d)
print("Ones e:\n", e)
print("Identity f:\n", f)
print("Random g:\n", g)

# 2 Array Properties
print("\nShape:", g.shape)
print("Dimensions:", g.ndim)
print("Size:", g.size)
print("Data type:", g.dtype)

# 3 Reshape & Transpose
reshaped = g.reshape(1, 9)
print("\nReshaped array:", reshaped)
print("Transpose:\n", g.T)
print("Flattened:", g.flatten())

# 4 Mathematical Operations
print("\nAdd:", np.add(a, 2))
print("Multiply:", np.multiply(a, 3))
print("Square Root:", np.sqrt(a))
print("Power:", np.power(a, 2))
print("Absolute:", np.abs([-1, -2, -3]))

# 5 Statistical Functions
print("\nMean:", np.mean(a))
print("Median:", np.median(a))
```

```
print("Standard Deviation:", np.std(a))
print("Variance:", np.var(a))
print("Minimum:", np.min(a))
print("Maximum:", np.max(a))
print("Sum:", np.sum(a))
print("Percentile (50):", np.percentile(a, 50))

# 6 Axis-based Operations
print("\nColumn-wise Sum:\n", np.sum(g, axis=0))
print("Row-wise Mean:\n", np.mean(g, axis=1))

# 7 Indexing & Slicing
print("\nFirst element:", a[0])
print("Slice:", a[1:4])
print("2D Slice:\n", g[0:2, 1:3])

# 8 Boolean & Conditional
print("\nWhere > 50:\n", np.where(g > 50))
print("Any > 90:", np.any(g > 90))
print("All > 0:", np.all(g > 0))

# 9 Sorting & Searching
print("\nSorted a:", np.sort(a))
print("Index of max:", np.argmax(a))
print("Index of min:", np.argmin(a))
print("Unique elements:", np.unique([1, 2, 2, 3, 4, 4]))

# 10 Linear Algebra
x = np.array([[1, 2], [3, 4]])
y = np.array([[5, 6], [7, 8]])

print("\nDot Product:\n", np.dot(x, y))
print("Matrix Multiplication:\n", np.matmul(x, y))
print("Determinant:", np.linalg.det(x))
print("Inverse:\n", np.linalg.inv(x))

# 11 Random Functions
np.random.seed(1)
rand_sample = np.random.choice(a, size=3)
print("\nRandom Choice:", rand_sample)

# 12 Stack & Concatenate
h1 = np.array([1, 2, 3])
h2 = np.array([4, 5, 6])
```

```
print("\nConcatenate:", np.concatenate((h1, h2)))
print("Vertical Stack:\n", np.vstack((h1, h2)))
print("Horizontal Stack:", np.hstack((h1, h2)))

# 13 Save & Load
np.save("sample_array.npy", a)
loaded = np.load("sample_array.npy")
print("\nLoaded Array:", loaded)
```

Output:-



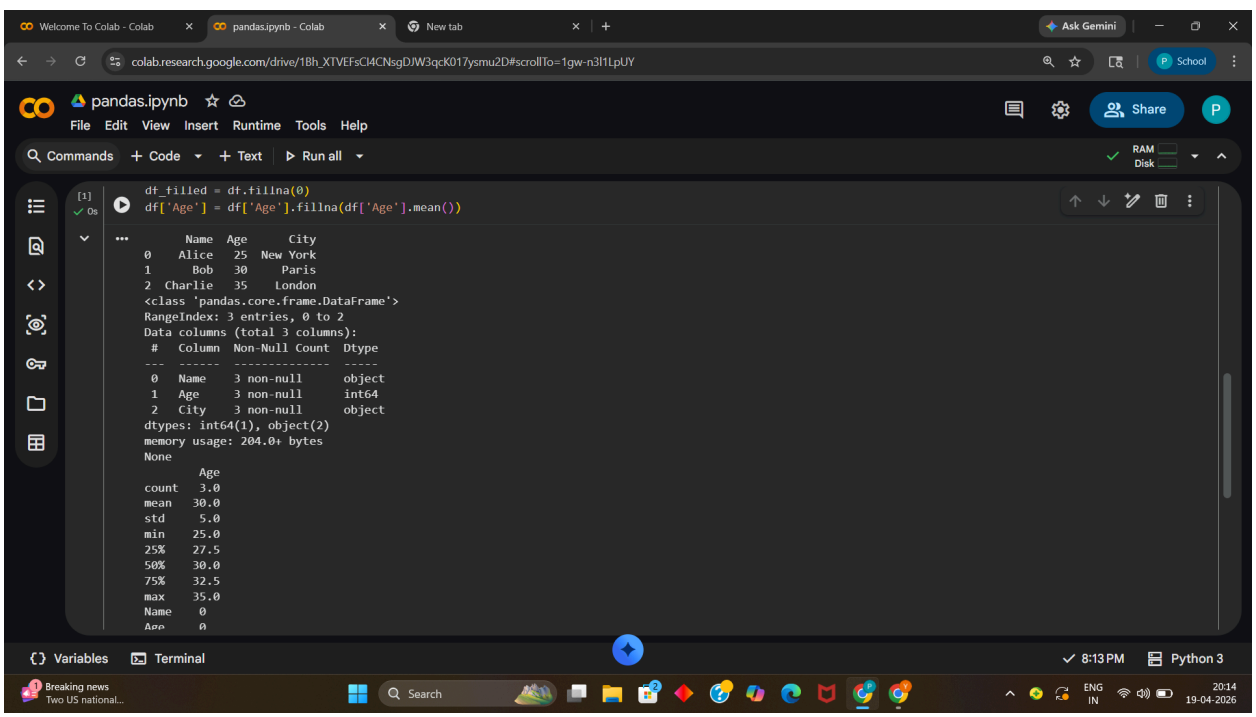


1.2 pandas(library):-

```
import pandas as pd

# A. Creating DataFrames
# From a Dictionary
data = {
    'Name': ['Alice', 'Bob', 'Charlie'],
    'Age': [25, 30, 35],
    'City': ['New York', 'Paris', 'London']
}

df = pd.DataFrame(data)

# Viewing data
print(df.head())        # First 5 rows
print(df.info())        # Summary of data types and non-nulls
print(df.describe())    # Statistical summary

# B. Selection and Filtering
# Selecting a column
ages = df['Age']
```

```
# Filtering rows
above_25 = df[df['Age'] > 25]

# iloc vs loc
row_0 = df.iloc[0]              # First row
specific_val = df.loc[0, 'City']   # 'New York'

# C. Data Cleaning
# Checking for nulls
print(df.isnull().sum())

# Dropping missing values
df_clean = df.dropna()

# Filling missing values
df_filled = df.fillna(0)
df['Age'] = df['Age'].fillna(df['Age'].mean())
```

Output:-

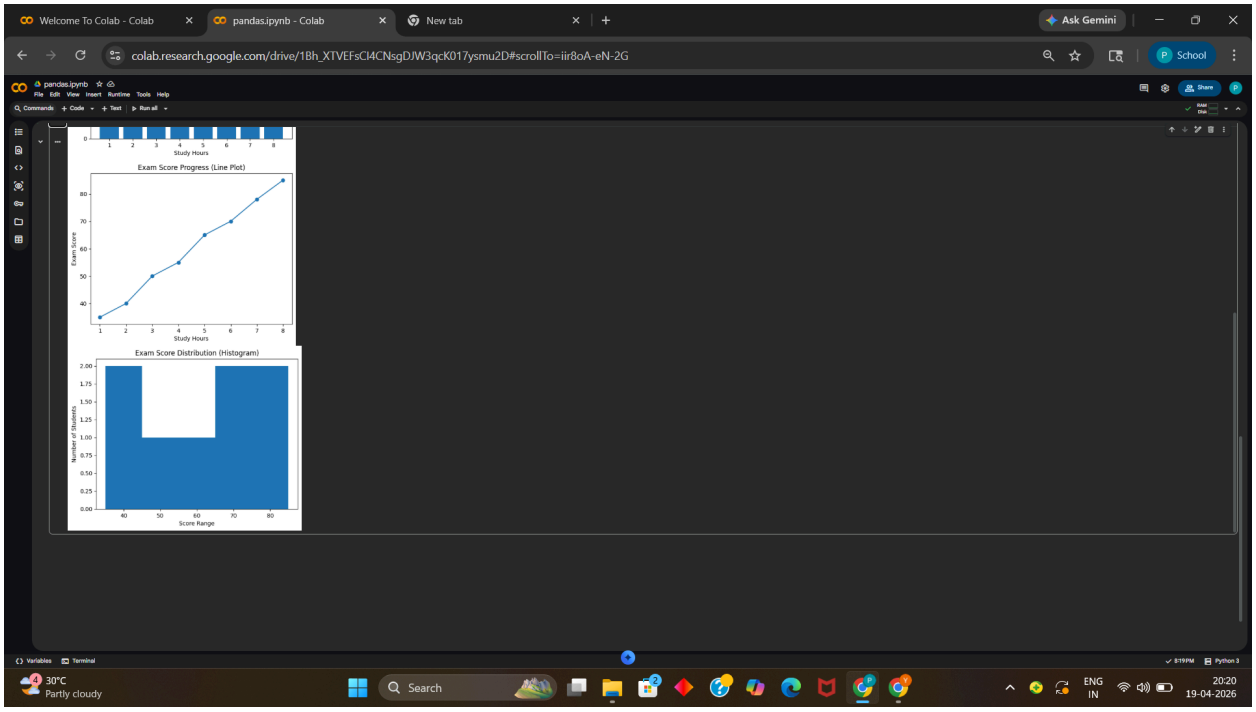


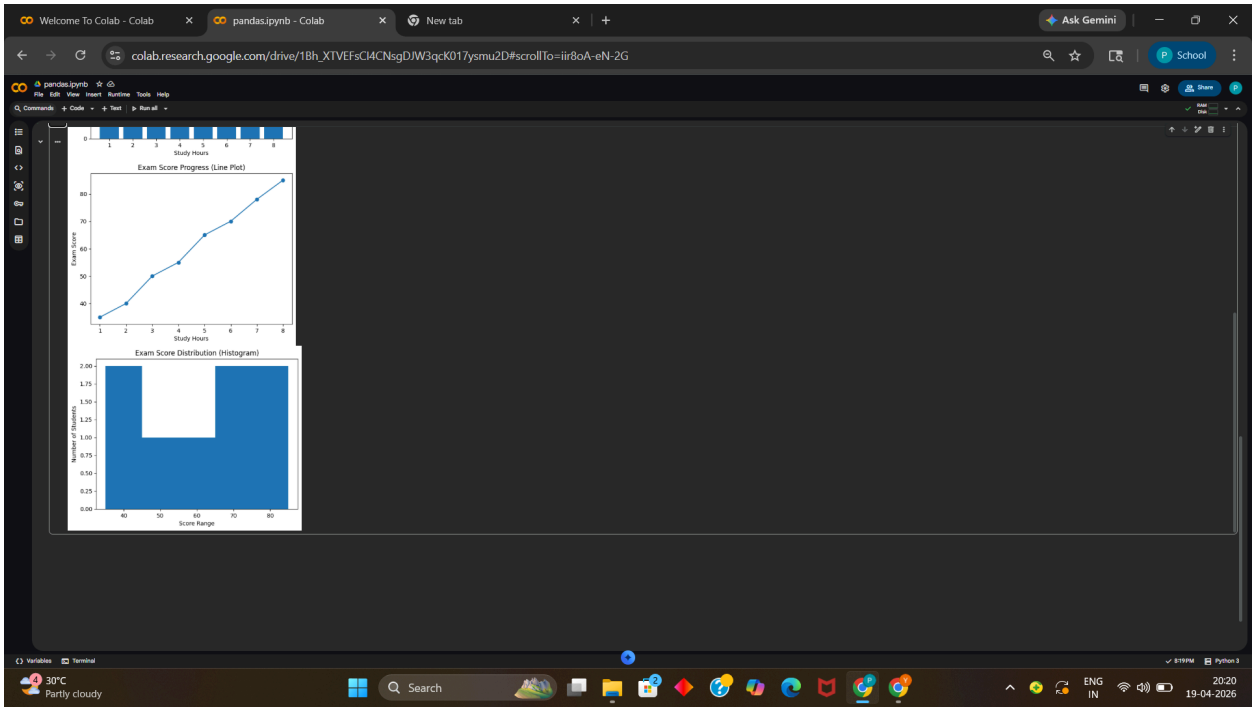
1.3 matplotlib(plotting library)

```
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np

# Create dataset
data = {
    'StudyHours': [1, 2, 3, 4, 5, 6, 7, 8],
    'ExamScore': [35, 40, 50, 55, 65, 70, 78, 85]
}
df = pd.DataFrame(data)

# -------------------------
# 1 Scatter Plot
# -------------------------
plt.scatter(df['StudyHours'], df['ExamScore'])
plt.xlabel("Study Hours")
plt.ylabel("Exam Score")
plt.title("Study Hours vs Exam Score (Scatter Plot)")
plt.show()

# -------------------------
# 2 Bar Chart
# -------------------------
plt.bar(df['StudyHours'], df['ExamScore'])
plt.xlabel("Study Hours")
plt.ylabel("Exam Score")
plt.title("Study Hours vs Exam Score (Bar Chart)")
plt.show()

# -------------------------
# 3 Line Plot
# -------------------------
plt.plot(df['StudyHours'], df['ExamScore'], marker='o')
plt.xlabel("Study Hours")
plt.ylabel("Exam Score")
plt.title("Exam Score Progress (Line Plot)")
plt.show()

# -------------------------
# 4 Histogram
# -------------------------
plt.hist(df['ExamScore'], bins=5)
```

```
plt.xlabel("Score Range")
plt.ylabel("Number of Students")
plt.title("Exam Score Distribution (Histogram)")
plt.show()
```

Output:-



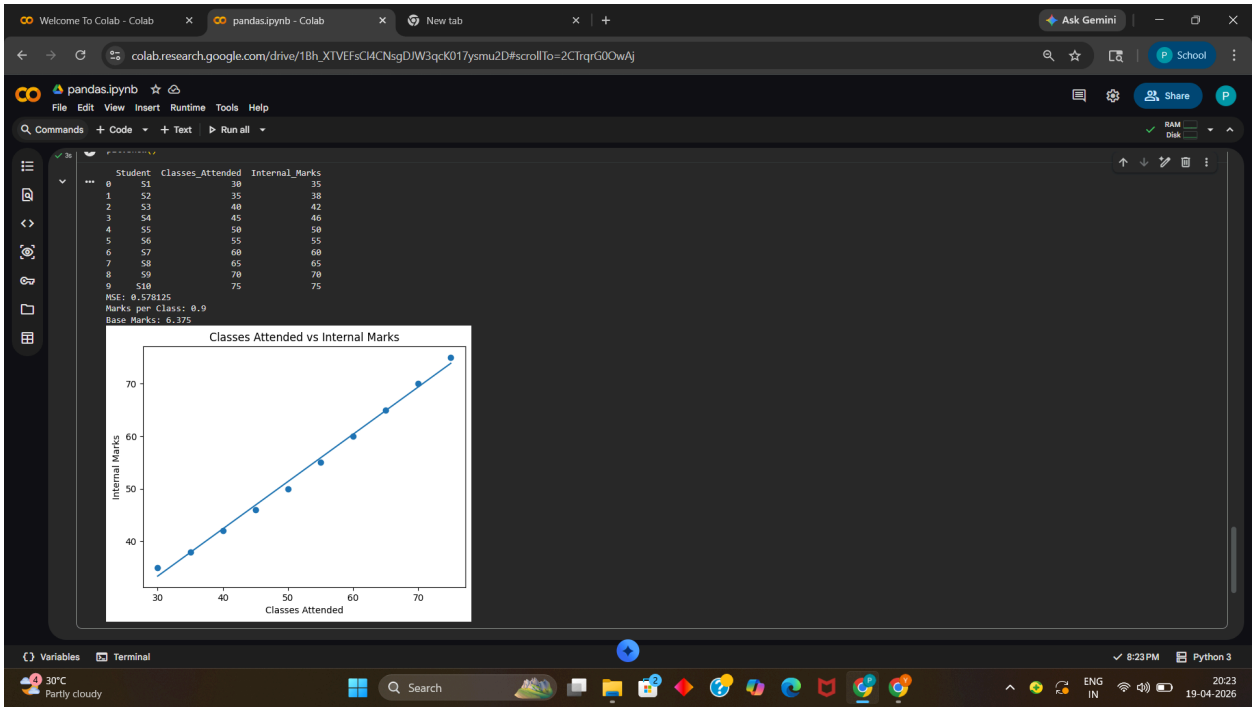


1.4 scikit - learn(machine learning)

```
# STEP 1: IMPORT LIBRARIES
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error

# STEP 2: CREATE DATASET
data = {
    'Student': ['S1','S2','S3','S4','S5','S6','S7','S8','S9','S10'],
    'Classes_Attended': [30, 35, 40, 45, 50, 55, 60, 65, 70, 75],
    'Internal_Marks': [35, 38, 42, 46, 50, 55, 60, 65, 70, 75]
}
df = pd.DataFrame(data)
print(df)

# STEP 3: FEATURES & TARGET
X = df[['Classes_Attended']]
y = df['Internal_Marks']

# STEP 4: TRAIN-TEST SPLIT
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=7
)

# STEP 5: TRAIN MODEL
model = LinearRegression()
model.fit(X_train, y_train)

# STEP 6: PREDICTION
predictions = model.predict(X_test)

# STEP 7: EVALUATION
mse = mean_squared_error(y_test, predictions)
print("MSE:", mse)
print("Marks per Class:", model.coef_[0])
print("Base Marks:", model.intercept_)

# STEP 8: VISUALIZATION
```

```
plt.scatter(X, y)
plt.plot(X, model.predict(X))
plt.xlabel("Classes Attended")
plt.ylabel("Internal Marks")
plt.title("Classes Attended vs Internal Marks")
plt.show()
```

Output:-

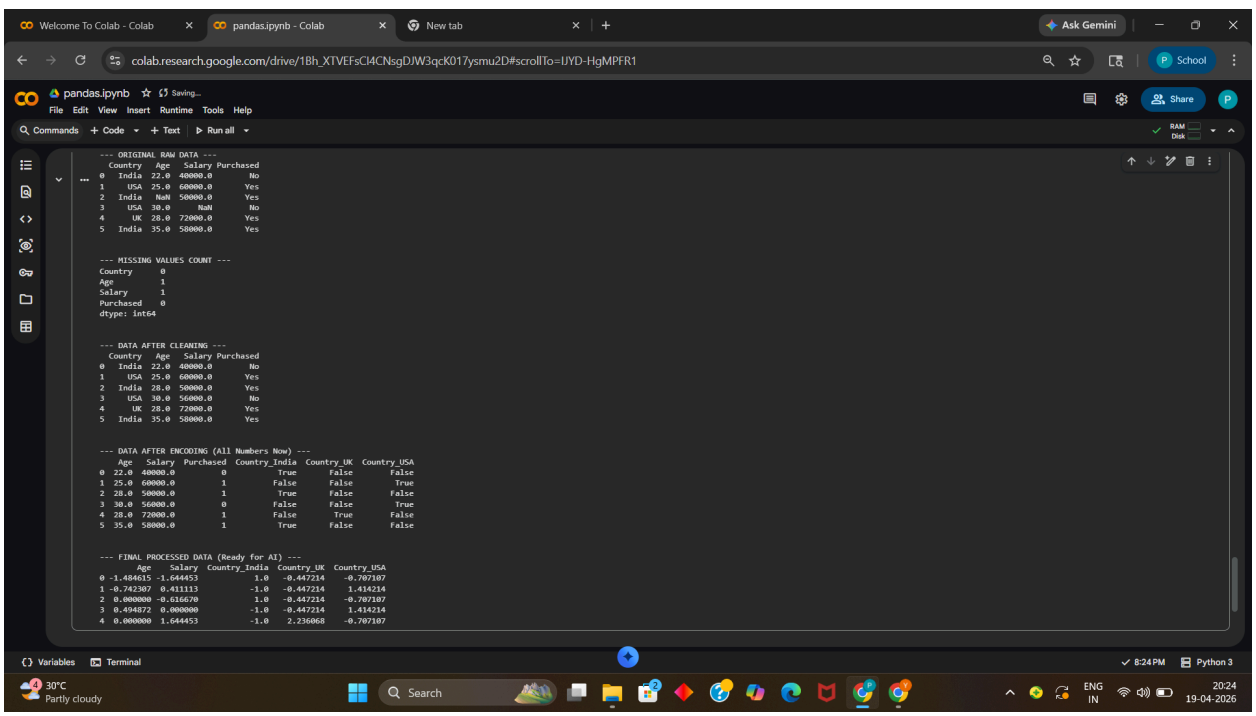


EX:-02
PERFORM DATA EXPLORATION and PREPROCCESING IN PYTHON

```
import pandas as pd
import numpy as np
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split

# --- STEP 1: LOAD DATA (Creating a dummy dataset for the lab) ---
data = {
    'Country': ['India', 'USA', 'India', 'USA', 'UK', 'India'],
    'Age': [22, 25, np.nan, 30, 28, 35],  # Note the 'nan' (missing value)
    'Salary': [40000, 60000, 50000, np.nan, 72000, 58000],
    'Purchased': ['No', 'Yes', 'Yes', 'No', 'Yes', 'Yes']
}
df = pd.DataFrame(data)
print("--- ORIGINAL RAW DATA ---")
print(df)
print("\n")

# --- STEP 2: INSPECT DATA ---
# Check for missing values
print("--- MISSING VALUES COUNT ---")
print(df.isnull().sum())
print("\n")

# --- STEP 3: CLEAN DATA (Handling Missing Values) ---
# Logic: Fill missing Age/Salary with the Average (Mean) of that column
df['Age'] = df['Age'].fillna(df['Age'].mean())
df['Salary'] = df['Salary'].fillna(df['Salary'].mean())
print("--- DATA AFTER CLEANING ---")
print(df)
print("\n")

# --- STEP 4: CONVERT TEXT TO NUMBERS (Encoding) ---
# Logic: Machines can't read 'India' or 'USA'. We convert them to 0s and 1s.
# We use 'One Hot Encoding' (get_dummies)
df_encoded = pd.get_dummies(df, columns=['Country'])

# For the Target column (Purchased), let's map Yes/No manually
df_encoded['Purchased'] = df_encoded['Purchased'].map({'Yes': 1, 'No': 0})
print("--- DATA AFTER ENCODING (All Numbers Now) ---")
print(df_encoded)
```

```
print("\n")

# --- STEP 5: SCALE FEATURES ---
# Logic: Age is 20-30, Salary is 40000-60000. Salary dominates Age because it's bigger.
# We shrink them to the same scale.
# Separate Features (X) and Target (y)
X = df_encoded.drop('Purchased', axis=1)
y = df_encoded['Purchased']

# Scale
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
print("--- FINAL PROCESSED DATA (Ready for AI) ---")
print(pd.DataFrame(X_scaled, columns=X.columns).head())
```

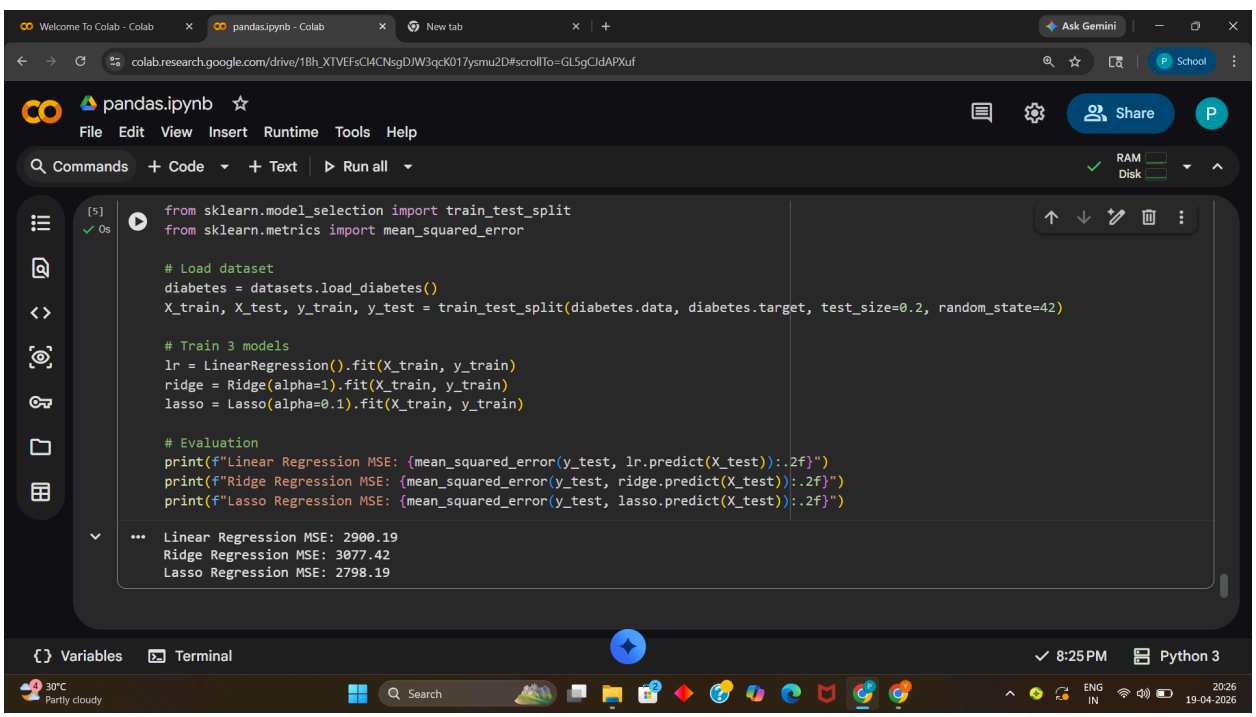
Output:-



EX:-03
Implement Regularized Linear Regression

```
from sklearn import datasets
from sklearn.linear_model import LinearRegression, Ridge, Lasso
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error

# Load dataset
diabetes = datasets.load_diabetes()
X_train, X_test, y_train, y_test = train_test_split(diabetes.data, diabetes.target, test_size=0.2,
random_state=42)

# Train 3 models
lr = LinearRegression().fit(X_train, y_train)
ridge = Ridge(alpha=1).fit(X_train, y_train)
lasso = Lasso(alpha=0.1).fit(X_train, y_train)

# Evaluation
print(f"Linear Regression MSE: {mean_squared_error(y_test, lr.predict(X_test)):.2f}")
print(f"Ridge Regression MSE: {mean_squared_error(y_test, ridge.predict(X_test)):.2f}")
print(f"Lasso Regression MSE: {mean_squared_error(y_test, lasso.predict(X_test)):.2f}")
```

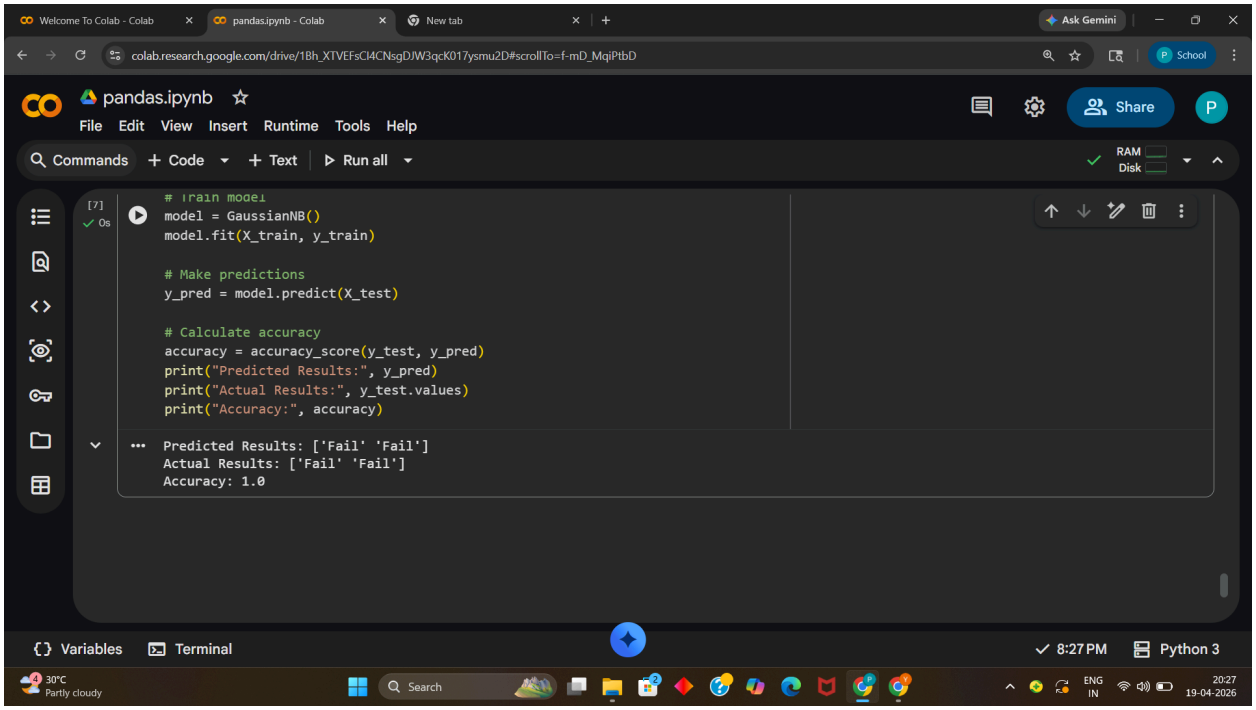
OUTPUT:-



EX:-0 4: Implement Naive Bayes Classifier

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score

# Create dataset manually
data = {
    'StudyHours': [1, 2, 4, 1, 6, 3, 7, 4],
    'Attendance': [30, 50, 60, 40, 85, 60, 90, 65],
    'Result': ['Pass', 'Fail', 'Pass', 'Fail', 'Pass', 'Fail', 'Pass', 'Fail']
}
df = pd.DataFrame(data)

# Features and target
X = df[['StudyHours', 'Attendance']]
y = df['Result']

# Split data into training and testing
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Train model
model = GaussianNB()
model.fit(X_train, y_train)

# Make predictions
y_pred = model.predict(X_test)

# Calculate accuracy
accuracy = accuracy_score(y_test, y_pred)
print("Predicted Results:", y_pred)
print("Actual Results:", y_test.values)
print("Accuracy:", accuracy)
```

Output:-

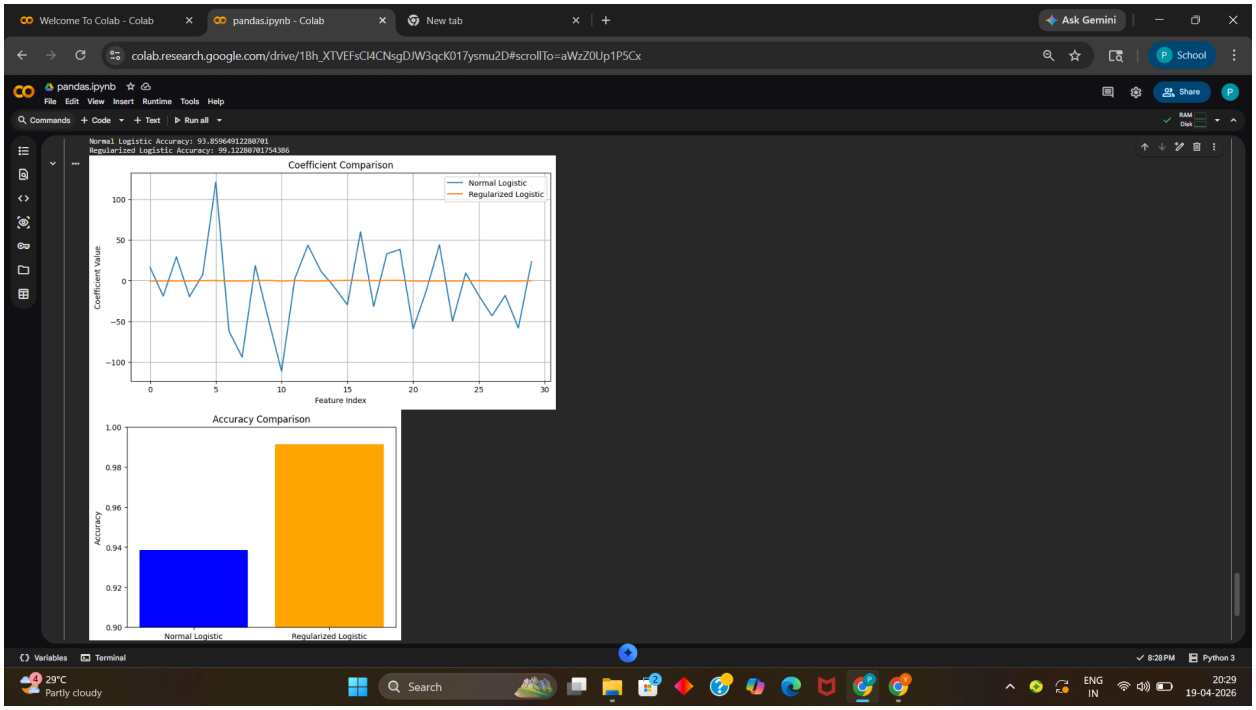


EX;-05
IMPLEMENT REGULARIZED LOGISTIC REGRESSION

```
# 1. Import Libraries
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

# 2. Load Dataset
data = load_breast_cancer()
X = data.data
y = data.target

# 3. Split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# 4. Scaling
```

```
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# 5. Normal Logistic Regression (Very Large C)
normal_model = LogisticRegression(C=1e6, penalty='l2', solver='liblinear')
normal_model.fit(X_train_scaled, y_train)
normal_pred = normal_model.predict(X_test_scaled)
normal_acc = accuracy_score(y_test, normal_pred)

# 6. Regularized Logistic Regression
regularized_model = LogisticRegression(C=0.1, penalty='l2', solver='liblinear')
regularized_model.fit(X_train_scaled, y_train)
reg_pred = regularized_model.predict(X_test_scaled)
reg_acc = accuracy_score(y_test, reg_pred)

# 7. Print Results
print("Normal Logistic Accuracy:", normal_acc * 100)
print("Regularized Logistic Accuracy:", reg_acc * 100)

# 8. Compare Coefficients
plt.figure(figsize=(10, 5))
plt.plot(normal_model.coef_.flatten(), label='Normal Logistic')
plt.plot(regularized_model.coef_.flatten(), label='Regularized Logistic')
plt.legend()
plt.title("Coefficient Comparison")
plt.xlabel("Feature Index")
plt.ylabel("Coefficient Value")
plt.grid()
plt.show()

# 9. Accuracy Comparison Bar Graph
plt.figure()
models = ['Normal Logistic', 'Regularized Logistic']
accuracies = [normal_acc, reg_acc]
plt.bar(models, accuracies, color=['blue', 'orange'])
plt.title("Accuracy Comparison")
plt.ylabel("Accuracy")
plt.ylim(0.9, 1.0)
plt.show()
```

Output:-

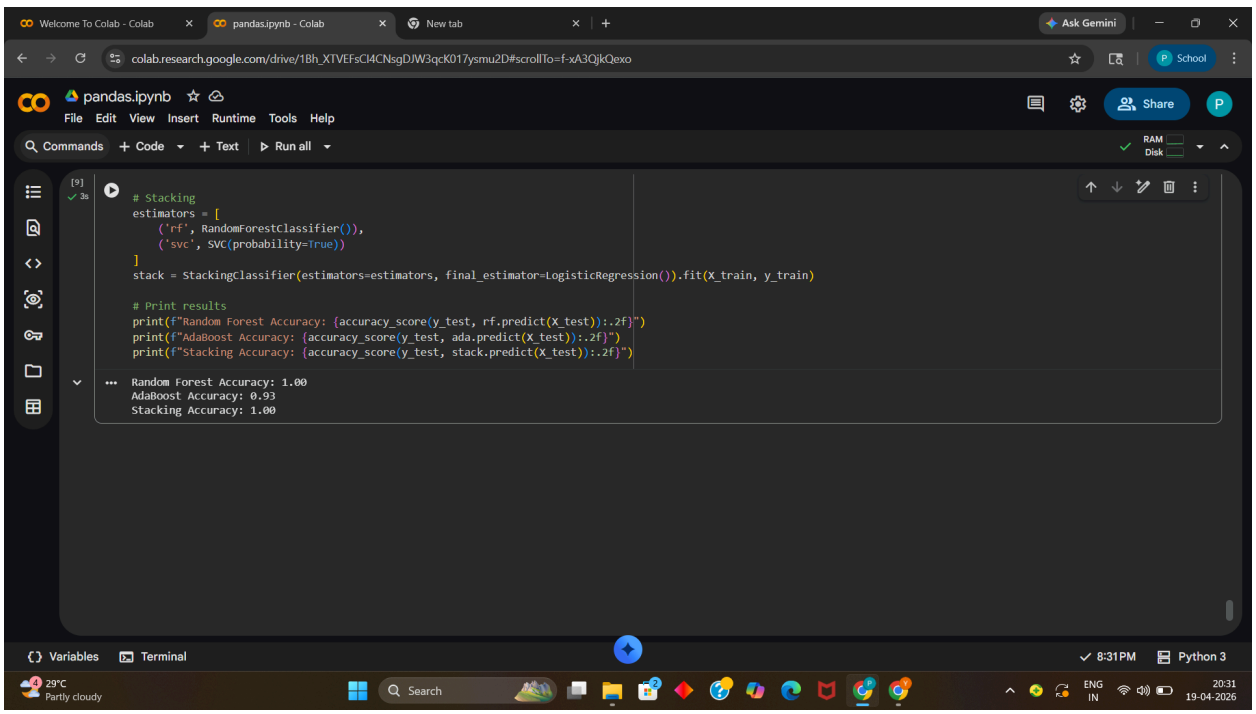


EX;-06.
Build Models using Different Ensembling Techniques

```
from sklearn import datasets
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier, AdaBoostClassifier, StackingClassifier
from sklearn.svm import SVC
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

# Load data
iris = datasets.load_iris()
X_train, X_test, y_train, y_test = train_test_split(iris.data, iris.target, test_size=0.2,
random_state=42)

# Bagging - Random Forest
rf = RandomForestClassifier(n_estimators=100).fit(X_train, y_train)

# Boosting - AdaBoost
ada = AdaBoostClassifier(n_estimators=100).fit(X_train, y_train)
```

```
# Stacking
estimators = [
    ('rf', RandomForestClassifier()),
    ('svc', SVC(probability=True))
]
stack = StackingClassifier(estimators=estimators,
final_estimator=LogisticRegression()).fit(X_train, y_train)

# Print results
print(f"Random Forest Accuracy: {accuracy_score(y_test, rf.predict(X_test)):.2f}")
print(f"AdaBoost Accuracy: {accuracy_score(y_test, ada.predict(X_test)):.2f}")
print(f"Stacking Accuracy: {accuracy_score(y_test, stack.predict(X_test)):.2f}")
```

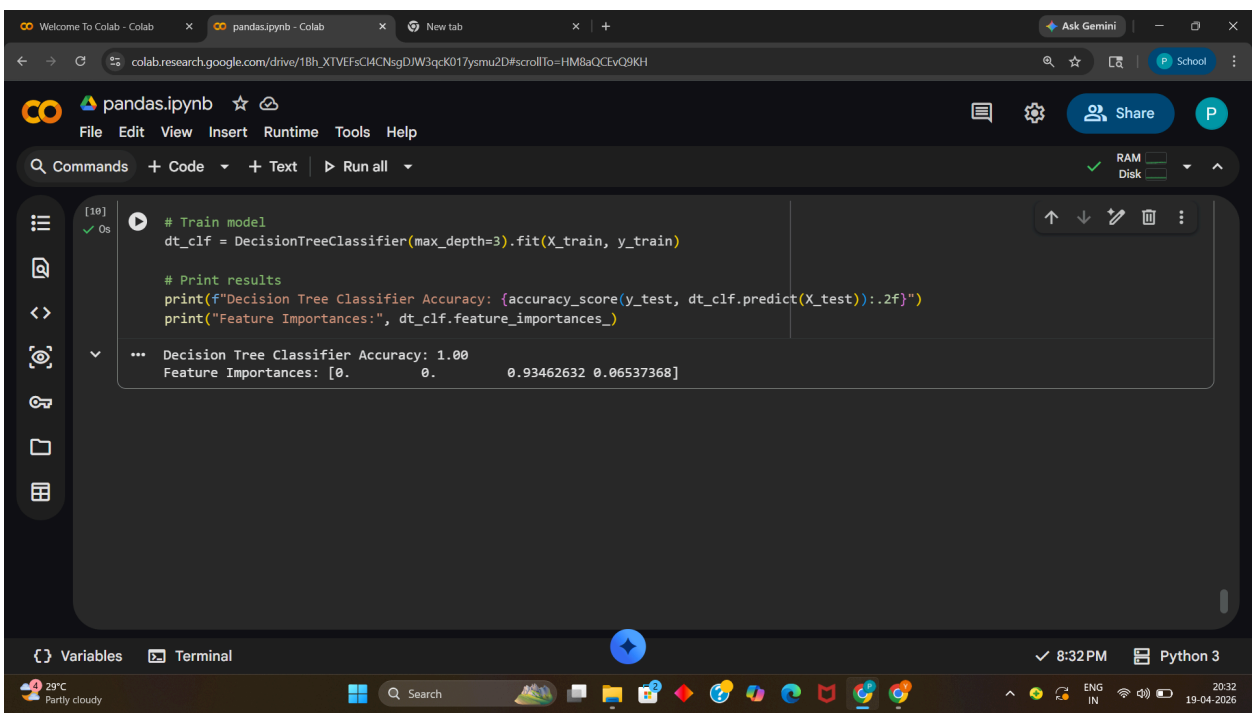
Output:-



EX-07
Build Models using Decision Trees

```
from sklearn import datasets
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier

# Load data
iris = datasets.load_iris()
X_train, X_test, y_train, y_test = train_test_split(iris.data, iris.target, test_size=0.2,
random_state=42)

# Train model
dt_clf = DecisionTreeClassifier(max_depth=3).fit(X_train, y_train)

# Print results
print(f"Decision Tree Classifier Accuracy: {accuracy_score(y_test, dt_clf.predict(X_test)):.2f}")
print("Feature Importances:", dt_clf.feature_importances_)
```

Output-

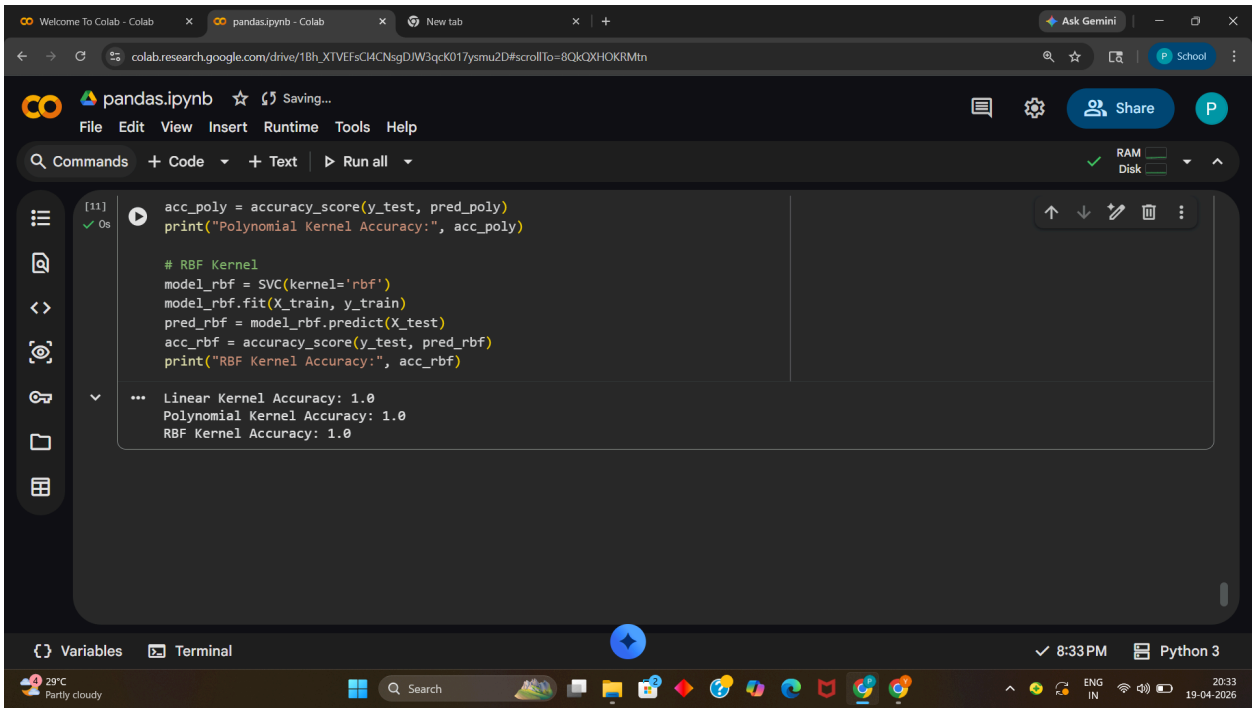


EX:-08

Support Vector Machine (SVM) with different kernels

```
# Import libraries
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score
from sklearn.datasets import load_iris

# Load dataset
data = load_iris()
X = data.data
y = data.target

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Linear Kernel
model_linear = SVC(kernel='linear')
model_linear.fit(X_train, y_train)
pred_linear = model_linear.predict(X_test)
acc_linear = accuracy_score(y_test, pred_linear)
print("Linear Kernel Accuracy:", acc_linear)

# Polynomial Kernel
model_poly = SVC(kernel='poly', degree=3)
model_poly.fit(X_train, y_train)
pred_poly = model_poly.predict(X_test)
acc_poly = accuracy_score(y_test, pred_poly)
print("Polynomial Kernel Accuracy:", acc_poly)

# RBF Kernel
model_rbf = SVC(kernel='rbf')
model_rbf.fit(X_train, y_train)
pred_rbf = model_rbf.predict(X_test)
acc_rbf = accuracy_score(y_test, pred_rbf)
print("RBF Kernel Accuracy:", acc_rbf)
```

Output-



Ex-09
Implement K-Nearest Neighbors (KNN) Classification

```
# Step 1: Import libraries
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

# Step 2: Load dataset
data = pd.read_csv("mobile_price.csv")

# Step 3: Define features and target
X = data.drop("price_range", axis=1)
y = data["price_range"]

# Step 4: Train-test split
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Step 5: Feature scaling
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Step 6: Create KNN model
knn = KNeighborsClassifier(n_neighbors=5)

# Step 7: Train model
knn.fit(X_train, y_train)

# Step 8: Prediction
y_pred = knn.predict(X_test)

# Step 9: Evaluation
print("Accuracy:", accuracy_score(y_test, y_pred))
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))
print("Classification Report:\n", classification_report(y_test, y_pred))
```

Output:-

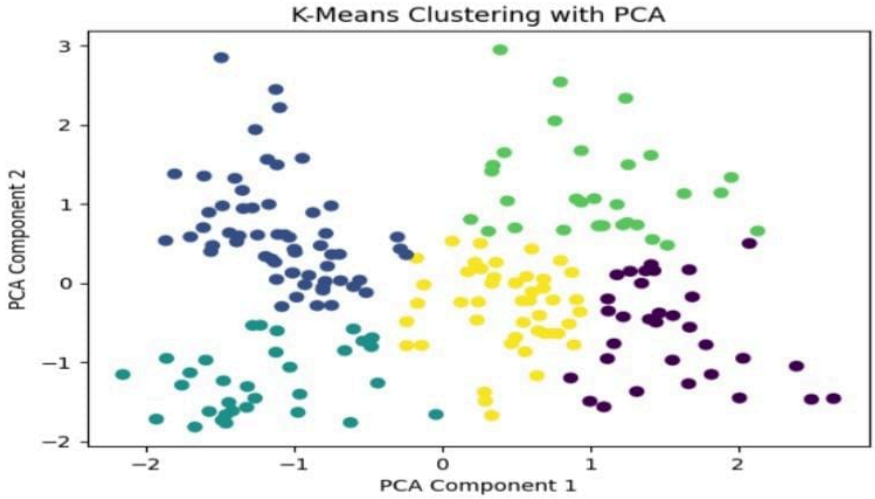
```
--- KNN Model Evaluation ---

Accuracy: 0.93

Confusion Matrix:
 [[95  5  0  0]
  [ 4 90  6  0]
  [ 0  5 88  7]
  [ 0  0  6 94]]

Classification Report:
              precision    recall  f1-score   support

           0       0.96      0.95      0.95       100
           1       0.90      0.90      0.90       100
           2       0.88      0.88      0.88       100
           3       0.93      0.94      0.94       100

    accuracy                           0.93       400
   macro avg       0.92      0.92      0.92       400
weighted avg       0.92      0.93      0.92       400
```

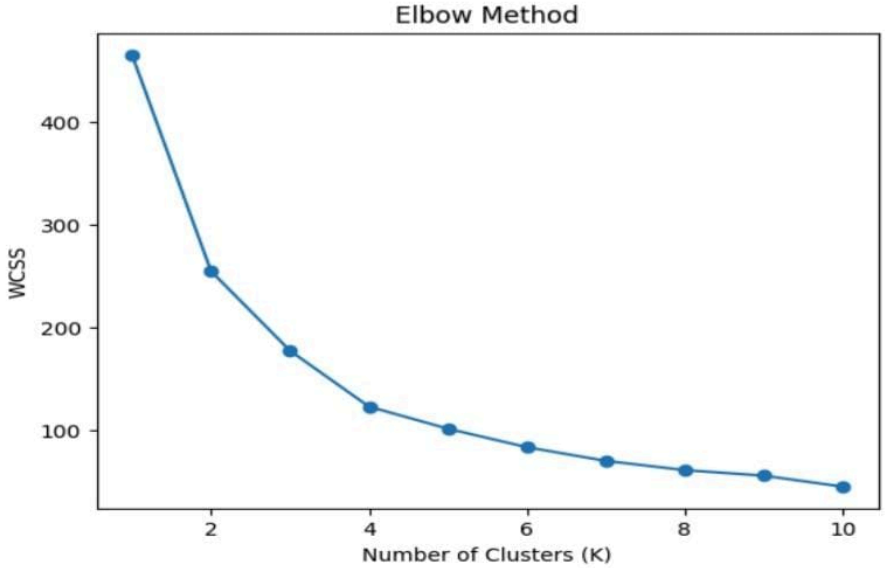
EX-10
K-means clustering with PCA and ELBOW method

```
# Import libraries
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.cluster import KMeans

# Load dataset
data = pd.read_csv("Mall_Customers.csv")

# Select numerical features
X = data[['Age','Annual Income (k$)','Spending Score (1-100)']]
```

```
# Feature scaling
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Apply PCA
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)
print("Explained Variance Ratio:", pca.explained_variance_ratio_)

# Elbow Method to find optimal K
wcss = []
for i in range(1, 11):
    kmeans = KMeans(n_clusters=i, init='k-means++', random_state=42)
    kmeans.fit(X_pca)
    wcss.append(kmeans.inertia_)

# Plot Elbow Graph
plt.plot(range(1,11), wcss, marker='o')
plt.title("Elbow Method")
plt.xlabel("Number of Clusters (K)")
plt.ylabel("WCSS")
plt.grid()
plt.show()

# Apply KMeans with optimal K (example K=5)
kmeans = KMeans(n_clusters=5, random_state=0)
clusters = kmeans.fit_predict(X_pca)

# Plot Clusters
plt.scatter(X_pca[:,0], X_pca[:,1], c=clusters, cmap='viridis')
plt.title("K-Means Clustering with PCA")
plt.xlabel("PCA Component 1")
plt.ylabel("PCA Component 2")
plt.show()
```

Output-


K-Means Clustering with PCA
PCA Component 2
PCA Component 1
3
2
1
0
−1
−2
−2
−1
0
1
2


Elbow Method
WCSS
Number of Clusters (K)
400
300
200
100
2
4
6
8
10